

Building a Cross-Platform Mobile Application

with Flutter

Building a mobile application is a necessity for every business and choosing the right tech stack for building the app is always a challenge. With so many options available in the market, why choose Flutter? Let's find out.



Launched in May 2017, Flutter is a free, open source, and cross-platform UI SDK created by Google to develop an application for Android and iOS, as well as web applications.

One of the first reasons to consider using Flutter is that you will end up with one code base to maintain, one place to debug, and one place to update your application. This is a huge advantage compared to creating your apps natively, say, building your iOS apps in Swift and your Android apps in Java or Kotlin. To update any feature in your application, you need to make changes at numerous places within it, which is both difficult

and error-prone. If you use Flutter, you only need to know one language — Dart, which is a powerful language and easy to work with. Once you have learnt and understood how to use it, you can create your iOS, Android, and web apps. You have to learn only one language instead of many.

If you have done any kind of programming before, then you will realise that Dart is actually very similar to a lot of modern object-oriented programming languages. It has been used at Google to build powerful tools such as Google AdWords and Google Fiber. I bet its usage is only going to get bigger in the future.

Anatomy of a Flutter app

Everything in the Flutter app is a widget. Just like Lego blocks, you put a widget upon another widget to build the application. To develop an app using Flutter, the first thing to do is to create a scaffold, i.e., a blank screen for our app. Inside this scaffold, we need to add an app bar at the top and a container below that. These are pre-built widgets. The app bar looks like the toolbar/app bar and acts like one. The container is nothing but a box that will contain the contents of the application. This will look somewhat like the Android UI design — scaffold as your parent layout, app bar as a toolbar, and container just